CSD457 INTERNET OF THINGS · GROUP G-04 · OCTOBER 2026

Progress report: Fleet-Orthogonal Adaptation, poison-resilient on-device intrusion detection for drifting IoT

Rishit Kamboj and Farhan Naik · What we built and measured, what it shows, and what the hardware will add.

The project in one paragraph

An IoT device that learns what normal traffic looks like can be taught, slowly, that attacks are normal; and a defence that stops learning when traffic looks suspicious can be frozen. From one device's traffic alone, honest change and slow poisoning cannot be told apart. Our method, **Fleet-Orthogonal Adaptation (FOA)**, uses the device's neighbours as a second witness: the part of an update that vulnerability-diverse peers also see is learned in full, the device-only rest is admitted through a budget that never closes. It is implemented, tested on all nine N-BaIoT devices and on 40 weeks of real CESNET traffic, written up as an IEEE-format paper, and ported to ESP32 firmware that is waiting for the boards.

Where we stand

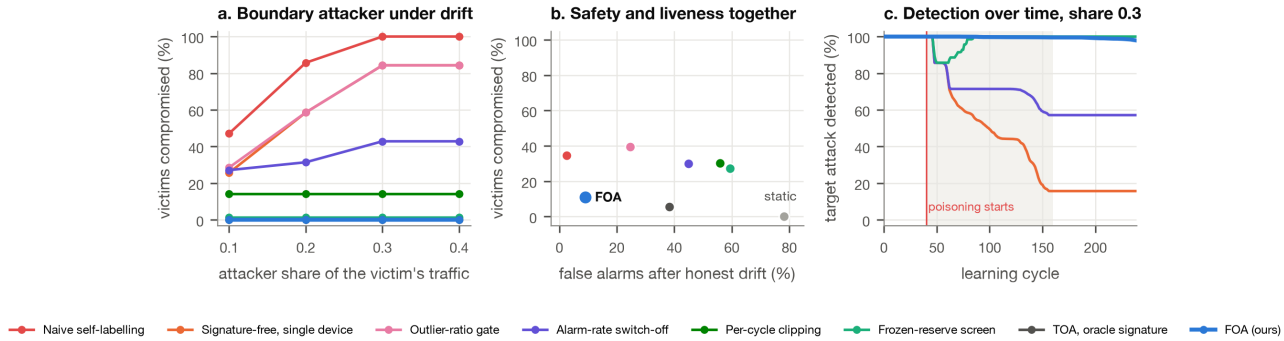
Area	Status	One line
Method (FOA) and theory	**Done**	Algorithm, four results with proofs (containment, Byzantine bound, frontier, no freeze state)
Simulation on real traffic	**Done**	All 9 N-BaIoT devices, 8 baselines, 12 attacker types, about 19,500 fleet runs, held-out seeds
Real drift	**Done**	40 weeks of CESNET traffic, 12 addresses of one institution
Freeze attack	**Done**	Shown on the alarm-rate switch-off; FOA provably and measurably immune
Paper	**Done**	paper/main.pdf, 11 pages, every number generated from the results
ESP32 firmware	**Built, not run**	Compiles; matches the Python reference on every recorded cycle; adds about 22 KB flash
ESP32 measurements	**Not yet**	Latency, energy and radio loss need the boards

Main results

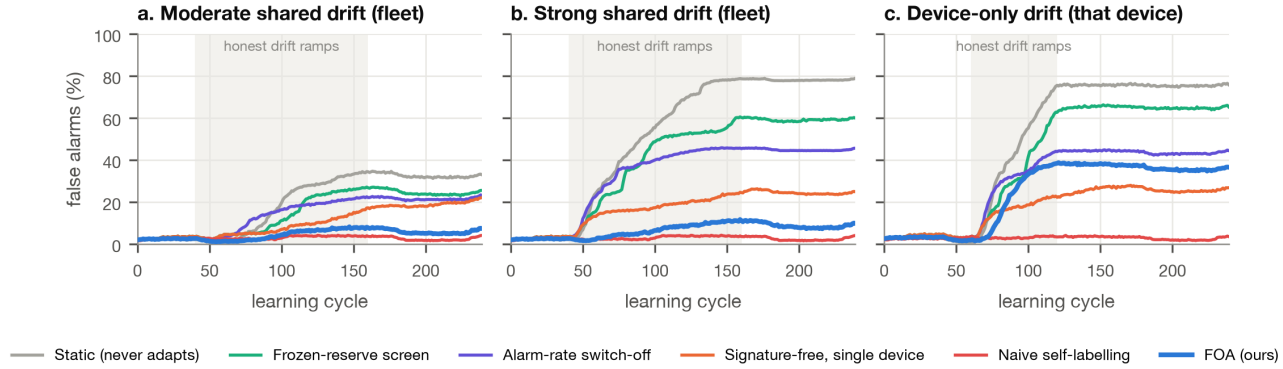
- **Following honest drift.** Under strong network-wide drift FOA ends at **9.0%** false alarms; a single device learning alone 24.6%, defences that pause learning 45% to 59%, a detector that never learns 78%.
- **Poisoning hidden inside drift.** Compromised overall: FOA **10.8%**, single device 39.2%. The classic slow poisoner never beat FOA (0%).
- **Monoculture botnet.** Same-model consensus (CIoTA style) 100% compromised once most cameras of a model are infected; FOA with diverse peers 0%.
- **Freeze attack.** 5% decoy traffic locks an alarm-rate switch-off at **100%** false alarms for good; FOA unchanged.
- **Real CESNET drift.** FOA compromised in 0% of runs, single device 18%.
- **Bound.** The proven minimum time to compromise held in every attacked FOA run (0 violations).

RESULTS

Two figures that carry the story



Poisoning hidden inside strong shared drift, 7 victims, 10 held-out seeds. (a) Boundary poisoner: FOA is never compromised. (b) FOA alone is low on both compromise and false alarms. (c) Detection of the target attack over time.



False alarms with no attacker. Shared drift is followed through the peers; drift on one device only is admitted at the budget rate, the trade-off the theory says no rule can avoid.

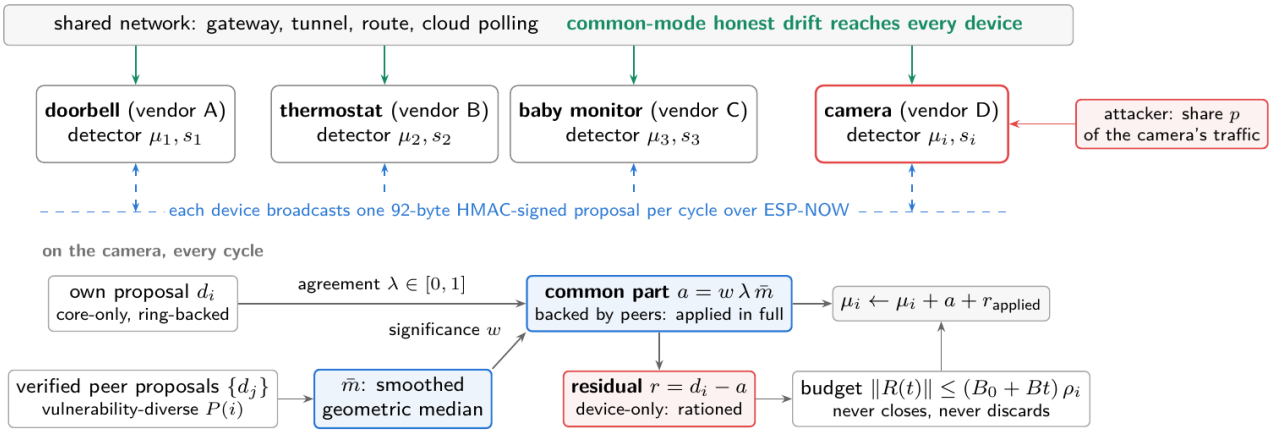
What goes against us (reported in the paper)

- A device whose own traffic changes a lot, on top of shared drift, is lost by FOA (single-device adaptation keeps it): device-only drift ends at 36% false alarms for FOA against 26%.
- Malicious peers alone never compromised anyone, but helping a traffic poisoner they beat borderline victims: 56% with three of seven peers.
- Core-hiding and mimicry poisoners at 40% share still win about a third of the time (34%, 29%).
- Fleet concurrency, the N-BaIoT drift and the poisoner are simulated; hardware numbers are estimates.

METHOD AND FIRMWARE

How FOA works and what is ready for the boards

In short: Learn freely what the neighbours also see; ration what only this device sees; never stop learning.



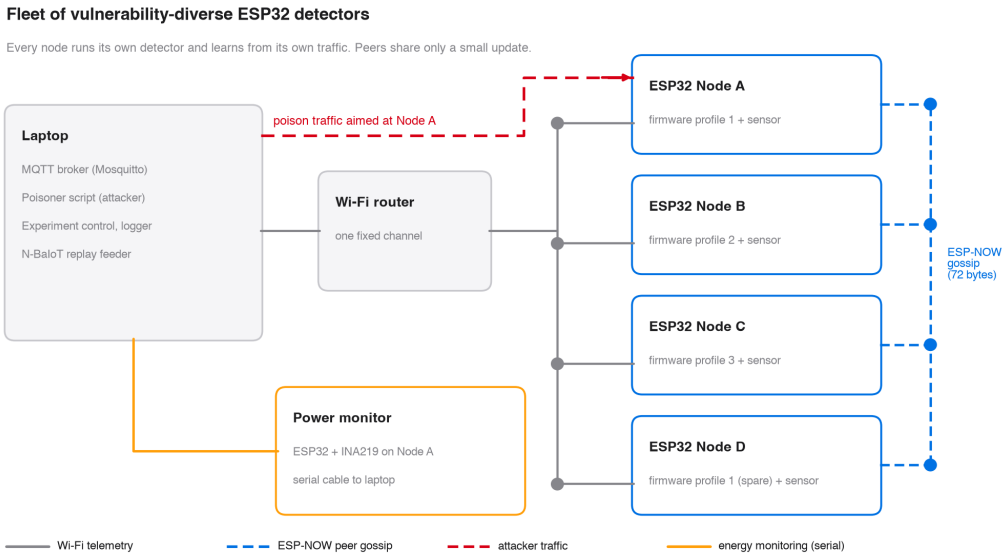
Firmware status

Check	Result
C core vs Python reference, 480 recorded cycles	0 mismatches
Tampered gossip frames accepted (bit flips, wrong keys, replays)	0 of 23,104
HMAC-SHA256, RFC 4231 test vector	pass
Node firmware for ESP32 (Arduino core 3.3.12)	915 KB flash, 69 KB RAM; FOA's own share about 22 KB / 20 KB
Power monitor (INA219, no library)	compiles; about 1 kHz samples
Laptop tools	poisoner (boiling frog, pulse, decoy, flood) and collector (latency, peers, mJ per cycle)

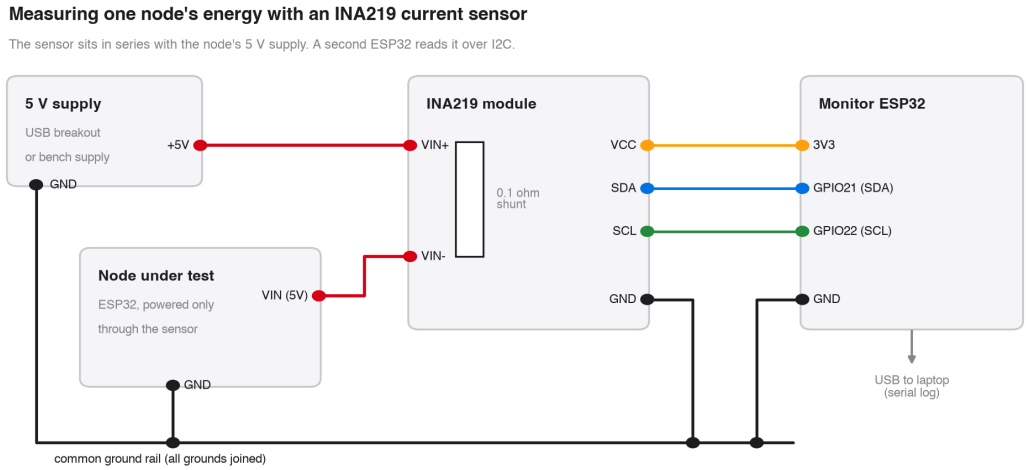
Hardware-day steps are in firmware/README.md: wire the INA219, fix the router channel, flash four nodes, record 20 minutes of clean traffic to fit the shared space, then run idle, drift, poisoning and freeze sessions.

HARDWARE

IoT set-up and wiring



Four ESP32 nodes with different firmware share a Wi-Fi network and gossip small signed updates over ESP-NOW. The laptop runs the MQTT broker, the poisoner and the logger. A separate ESP32 reads the INA219 on Node A.



Build notes (check before wiring):
1. Add 4.7 kohm pull-ups from SDA and SCL to 3V3 if the INA219 module has none.
2. Flash the node first, then power it only through this path. Do not also plug in its USB at the same time.
3. INA219 samples about every 0.5 ms, so measure energy over many cycles and average. Do not trust single short bursts.

Proposed wiring, not yet built. Check every pin against your module's datasheet before powering it. Default I2C pins on the ESP32 DevKitC are GPIO21 (SDA) and GPIO22 (SCL).

PARTS AND PLAN

What we need, and what comes next

Item	Qty	Why
ESP32 development boards (for example ESP32-DevKitC)	4, plus 1 for monitoring	Fleet nodes A to D and the power monitor
INA219 current-sensor module, 5 V supply or USB breakout	1 to 2	Measures a node's power and energy
BME280 or DHT22 sensor (optional)	1 per node	Gives each node real telemetry to send
Breadboard, jumpers, 4.7 kohm resistors, USB cables and hub	1 set	Wiring, I2C pull-ups, flashing, serial logs
Wi-Fi router on a fixed channel; laptop	1 each	ESP-NOW and Wi-Fi share one channel; laptop runs broker, poisoner, logger

Software: ESP-IDF or the Arduino core (ESP-NOW and mbedTLS HMAC are standard), Python and NumPy, Mosquitto, and our experiment scripts. Two run modes: **replay** (N-BaIoT features fed over serial, for reliable cost numbers) and **live** (the board's own telemetry). Measured on the board: peak RAM and flash, latency per update and per signed message, ESP-NOW bytes per learning event, and energy per update from the INA219. The firmware already logs the time of every stage; our current figure (about 2 ms of compute per 10 s learning cycle) is an estimate until we measure it.

Next steps

When	Work	Output
Boards arrive	Flash four nodes and the monitor; fit the shared basis on clean testbed traffic; idle baseline	Real latency, RAM, bytes, energy per cycle
Week after	Real shared drift (a router or telemetry change on every node); laptop poisoner; decoy freeze test	FOA on real hardware with real drift
Before submission	Independent code review; read the three unread preprints; replace estimates in the paper with measurements	Final paper and demo

Risks and fallbacks

Risk	Fallback
Novelty is partial (CIoTA is close); several 2026 papers are preprints	We beat CIoTA-style consensus head to head; claims say 'we found no prior work', never 'first'
Simulated concurrency and drift on N-BaIoT	CESNET supplies real drift; the testbed supplies a real shared change
ESP-NOW and Wi-Fi on one channel misbehave	Fall back to UDP gossip through the router (same frames, same HMAC)

REFERENCES

Compiled references, newest first

36 references, 26 from 2025 to 2026. Full list with links, reading status and one line on why each matters: **references.md** *in the repository.* **Read** *says how much we actually read; preprints are not peer-reviewed, so check numbers against the PDF.*

Year	Paper	Type	Read
2026	Feature-Context Consistency	preprint	full
2026	The Price of Self-Calibration	preprint	not read
2026	Navigating the Latent Manifold	preprint	full
2026	CAPTURE: Disentangling Preference Drift from Memory Poisoning in Personaliz...	preprint	not read
2026	Candidate Comparability Before Promotion	preprint	abstract
2026	Empirical Analysis of Evasion and Poisoning Against Malware Data Drift Dete...	preprint	full
2026	Self-Poisoning in Adaptive Out-of-Distribution Detection	preprint	full
2026	Theory of Continual Learning Against Data Poisoning Attacks	preprint	full
2026	Rapid Poison: Practical Poisoning Attacks Against the Rapid Response Framework	peer-reviewed	full
2026	What changes after deployment? A survey on On-device Learning in TinyML	preprint	full
2026	When Normality Shifts: Risk-Aware Test-Time Adaptation for Unsupervised Tab...	preprint	full
2026	The Boiling Frog Threshold	preprint	full
2026	ACORN-IDS: Adaptive Continual Novelty Detection for Intrusion Detection Sys...	preprint	full
2026	Online robust covariance matrix estimation and outlier detection	preprint	full
2026	Contrastive Adversarial Agents for Intentional Drift Induction in Network I...	peer-reviewed	full
2026	Strategic Games and Zero Shot Attacks on Heavy-Hitter Network Flow Monitoring	peer-reviewed	full
2025	Provably Safe Model Updates	preprint	full
2025	Adversarial Habituation Attack	peer-reviewed	abstract
2025	Shift Detection and Adaptation for Network Intrusion Detection	preprint	full
2025	CITADEL: Continual Anomaly Detection for Enhanced Learning in IoT Intrusion...	preprint	not read
2025	CND-IDS: Continual Novelty Detection for Intrusion Detection Systems	peer-reviewed	full
2025	On the Adversarial Risk of Test Time Adaptation	peer-reviewed	full
2025	Provably Reliable Conformal Prediction Sets in the Presence of Data Poisoning	peer-reviewed	partial
2025	Adversarial Attacks for Drift Detection	peer-reviewed	partial
2025	**CORE** CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detect...	peer-reviewed	dataset
2025	Network Intrusion Datasets	peer-reviewed	partial
2024	Data Poisoning Attack against Neural Network-Based On-Device Learning Anoma...	peer-reviewed	full
2024	AOC-IDS: Autonomous Online Framework with Contrastive Learning for Intrusio...	peer-reviewed	full
2023	Anomaly Detection in the Open World	peer-reviewed	full
2022	Adversarial concept drift detection under poisoning attacks for robust data...	peer-reviewed	full
2018	CIoTA: Collaborative IoT Anomaly Detection via Blockchain	peer-reviewed	partial
2018	**CORE** N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders	peer-reviewed	dataset
2018	**CORE** Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection	peer-reviewed	full
2017	Machine Learning with Adversaries	peer-reviewed	full
2012	**CORE** Security Analysis of Online Centroid Anomaly Detection	peer-reviewed	full
2009	**CORE** ANTIDOTE: Understanding and Defending against Poisoning of Anomaly Detectors	peer-reviewed	full